

Grip Security Cluster Watchdog

Proactive Monitoring Agent — Design Brief

Author: Leon Kuperman | Date: June 3, 2026 | Status: Draft

Summary

We will build a lightweight monitoring agent that inspects Grip Security's clusters every 5 minutes, uses an LLM to distinguish genuine anomalies from normal operational noise, and posts actionable diagnostics to a shared Slack channel. The goal is to catch the class of issues we've seen in the last two incidents—OOMKill spirals, misapplied resource recommendations, stuck pods, agent failures—before they escalate to production fires. The agent runs as a scheduled job, calls CAST AI and Kubernetes APIs to collect a cluster snapshot, feeds that snapshot to Claude Haiku as a single-shot evaluation, and posts findings to Slack only when something looks genuinely wrong. This replaces the current reactive model where problems surface hours or days later via customer complaints.

Problem

Grip Security has experienced at least two major incidents in the last two weeks. The first was an integer overflow in the Workload Autoscaler that silently reduced memory limits from 143 GiB to 128 MiB, causing sustained OOMKills across multiple namespaces for four days before detection. The second was an OOMKill recovery mechanism that overshot resource requests beyond any available instance size, preventing workloads from scheduling entirely. In both cases, the failure was silent from CAST AI's perspective—our safety mechanisms didn't trigger, our event logs didn't surface the issue, and we learned about the problem from the customer.

The customer's CTO is questioning our value. Their team feels they spend too much time configuring the platform and handling incidents. We committed in an onsite meeting (June 1) to deliver better alerting and monitoring this week. This is that deliverable.

Tenets

Customer application stability over cost savings—we flag anything that threatens uptime, even if it means the customer temporarily runs at higher cost. Reducing false positives over catching every edge case—a noisy alerting channel gets muted within a week, so the agent must learn what's normal for these specific clusters before alarming. Transparency over polish—we show the customer exactly what data we looked at and why we flagged something, rather than hiding behind generic alerts.

Architecture

How It Works

The agent is a scheduled job that runs every 5 minutes. Each run executes three phases: collect, evaluate, and notify.

Collect: We already have a CAST AI internal MCP server (castai-remote) that provides direct access to cluster snapshots without needing kubectl. The collector calls `analyze_snapshot_with_code` to run Python against the live snapshot, which exposes `snapshot.get_pods()`, `snapshot.get_nodes()`, `snapshot.get_deployments()`, `snapshot.get_pdbes()`, `snapshot.get_hpas()`, and the full `snapshot.raw` dict (containing `podList`, `deploymentList`, `nodepoolv1List`, etc.). It also calls `woop_get_workloads` to get WOOP recommendation state and intended vs. applied values, `loki_query` to pull evictor and workload autoscaler logs, and `list_clusters` for org-wide enumeration. No kubectl or kubeconfig required—the MCP handles authentication via the CAST AI API token. The script maintains a rolling window of the last 12 snapshots (1 hour) in a local file so the evaluator can see trends.

Evaluate: The snapshot (current + trailing hour) is fed to Claude Haiku (or Sonnet for higher-fidelity evaluation) as a single-shot prompt. The prompt instructs the model to act as an SRE reviewing the cluster, identify anything that looks abnormal, classify findings by severity, and output a structured JSON response with a verdict: HEALTHY, WARNING, or CRITICAL. The model is given the cluster’s baseline characteristics (expected workloads, known scale-to-zero patterns, typical resource ranges) so it can distinguish “aggregator scaled to zero again” from “discovery-puller OOMKilling in a loop.”

Notify: If the verdict is WARNING or CRITICAL, the agent posts a formatted message to the shared Slack channel. The message includes: severity, affected workloads, what looks wrong, what data supports the conclusion, and a suggested action. If the verdict is HEALTHY, nothing is posted—silence means everything is fine. A daily summary is posted at 08:00 UTC regardless of status.

Data Sources — CAST AI MCP Tools

We have an internal MCP server (castai-remote) that provides all the data we need without requiring kubectl access to the customer cluster. The SE should use these tools directly:

MCP Tool	What It Returns	Use In Watchdog
analyze_snapshot_with_code	Runs arbitrary Python against the live cluster snapshot. Exposes snapshot.get_pods(), get_nodes(), get_deployments(), get_pdb(), get_hpas(), and snapshot.raw dict with podList, deploymentList, etc.	Primary data source. Run Python to extract pod status, OOMKill counts, restart counts, resource requests/limits, node capacity, CAST AI agent pod health — all in one call
woop_get_workloads	WOOP workload list with recommendation status, intended vs. applied resource values, optimization state, confidence scores	Detect recommendation/actual mismatches, data gaps (no recommendation), disabled optimizations, and multiplier configs
loki_query	Query evictor, workload autoscaler, and cluster controller logs from Loki	Pull WA mutation logs to detect overflow patterns, evictor blocked-node reasons, OOMKill safety mechanism events
list_clusters	All clusters in an org with status and agent health	Enumerate clusters to monitor, detect agent-offline conditions at the org level
get_cluster_snapshot_summary	High-level cluster summary: node count, pod count, cloud provider, region, K8s version	Cluster identity and metadata for context in the LLM prompt
extract_fields_from_all	Extract specific JSON fields from all resources of a given kind (e.g., all node labels, all pod statuses)	Fallback if analyze_snapshot_with_code times out — extract fields individually and compute locally

No kubectl, kubeconfig, or direct cluster network access is needed. The MCP authenticates via the customer’s CAST AI API token, which we already have.

Snapshot Python API

Inside analyze_snapshot_with_code, the snapshot object provides these methods. The SE should use these directly in the collector Python code:

```
# Pod-level data (OOMKills, restarts, resource specs)
pods = snapshot.get_pods()
```

```

for pod in pods:
    ns = pod['metadata']['namespace']
    for cs in pod.get('status', {}).get('containerStatuses', []):
        restarts = cs['restartCount']
        terminated = cs.get('lastState', {}).get('terminated', {})
        oom = terminated.get('reason') == 'OOMKilled'

# Node-level data (capacity, utilization)
nodes = snapshot.get_nodes()

# Deployment-level (replica counts, strategy)
deployments = snapshot.get_deployments()

# Raw access for anything else
vpa_items = snapshot.raw.get('verticalpodautoscalerList',
                             {}).get('items', [])

```

What the Agent Catches

Scenario	Detection Signal	Example from Grip
OOMKill spiral	Pod restart count increasing across consecutive snapshots, OOMKilled exit code	discovery-puller in cengage: 128 MiB limit, continuous OOMKills for 4 days
Recommendation/actual mismatch	WOOP intended recommendation differs from pod's actual resource spec by >50%	Int64 overflow: recommended 143 GiB, applied 128 MiB
Unschedulable workloads	Pods in Pending state with insufficient resource events	Recovery mechanism requested resources larger than any node
Agent down	CAST AI agent pod not Running or last heartbeat >10 min ago	N/A (preventive)
Data gap / no recommendation	Workload has optimization enabled but no active recommendation for >2 hours	aggregator in oscarhealth post-fix: no metrics, no recommendation
Memory leak pattern	Memory usage trending up across snapshots without stabilizing	aggregator workloads with unbounded memory growth
Absurd recommendation	WOOP recommendation exceeds 100 GiB memory or 100 CPU cores for a single workload (via <code>woop_get_workloads</code>)	June 1: discovery-puller in athenahealth recommended 10 TB RAM after OOM recovery logic compounded with Max Usage strategy
Agent restart loop	CAST AI agent pod <code>restartCount</code> >3 in last hour, or <code>ExitCode:0</code> with multiple recent restarts	March 18: castai-agent exiting with code 0 silently for days across all prod clusters. May 25: agent restarting 11+ times on prod-us-3

Webhook / exporter failure	Workload autoscaler admission webhook not responding, or exporter hitting metric collection limits	April 1: WA webhook non-functional on prod-eu. Exporter metric limit hit on prod4 causing incomplete recommendations
Cascading scaling	Rapid node count or pod count increase (>50% in 30 min) without corresponding workload deployment	April 3: cluster running at double expected nodes (~3000 pods) for 3+ hours undetected

Sample Evaluation Prompt

The following is the complete prompt sent to Claude Haiku/Sonnet every 5 minutes. The {SNAPSHOT_JSON} placeholder is replaced with the collected cluster data.

SYSTEM PROMPT FOR CLUSTER WATCHDOG EVALUATION

You are an SRE monitoring agent for CAST AI. You inspect Kubernetes cluster snapshots every 5 minutes and identify anomalies that need human attention. Your job is to reduce noise—only flag things that a senior SRE would actually investigate.

Cluster Context (Grip Security — prod-us-4)

Known baseline behaviors (DO NOT flag these as anomalies):

- aggregator workloads in every namespace routinely scale to zero replicas. Seeing 0/0 ready pods for aggregator is NORMAL.
- discovery-puller workloads have high memory usage (800MB-1.5GB per pod is expected). Flag only if limits are set far below this.
- Some namespaces are low-traffic and may show 0 resource usage. This is not an incident.

Known configuration:

- WOOP memory limit multiplier: 1.5x (recently corrected from 100x)
- WOOP recommendation strategy: Max Usage (NOTE: this strategy is vulnerable to outlier pods driving extreme recommendations — flag any recommendation where a single pod's usage is >10x the median of its siblings as a potential outlier-driven spike)
- Cluster ID: e9f502e2-46dd-49fb-9310-149d7d8ad0ba
- Multiple customer namespaces (cengage, williamsmullen, oscarhealth, ensemblehealthpartners, athenahealth, and others)
- Cilium CNI in use — nodes may briefly show cilium agent-not-ready taint during startup. This is normal and transient (<2 min).
- Customer uses KEDA for some workloads — rapid replica changes driven by KEDA are expected and should not trigger cascading scaling alerts unless node count also spikes disproportionately.

Known fragile components (extra scrutiny):

- castai-agent: has a history of ExitCode:0 silent failures and OOM issues. Any restart is worth logging; >3/hour is CRITICAL.
- castai-workload-autoscaler: should be running with 2 replicas. If only 1 replica is Running, flag as WARNING.
- castai-workload-autoscaler-exporter: watch for metric limit errors in logs (via loki_query). If exporter is unhealthy, recommendations will be incomplete.

Your Task

Analyze the following snapshot and trailing history. Produce a JSON response with this exact structure:

```
{
  "verdict": "HEALTHY" | "WARNING" | "CRITICAL",
  "summary": "<one sentence overall assessment>",
  "findings": [
    {
      "severity": "critical" | "warning" | "info",
      "category": "<oomkill|mismatch|unschedulable|agent|data_gap|
        memory_leak|absurd_recommendation|agent_restart|
        webhook_failure|cascading_scaling|config|other>",
      "workload": "<namespace/workload or cluster-level>",
      "what": "<what is happening>",
      "evidence": "<specific numbers from the snapshot>",
      "suggested_action": "<what the TAM or customer should do>"
    }
  ],
  "metrics": {
    "total_pods": <int>,
    "oomkilled_pods": <int>,
    "pending_pods": <int>,
    "crashloop_pods": <int>,
    "oomkill_rate_per_hour": <float>,
    "recommendation_mismatches": <int>,
    "absurd_recommendations": <int>,
    "agent_restarts_last_hour": <int>,
    "node_count": <int>,
    "node_count_delta_pct": <float>
  }
}
```

Classification Rules

CRITICAL (post immediately):

- Any pod OOMKilled more than 3 times in the last hour
- Any WOOP recommendation where applied value differs from intended by more than 50%
- Any workload with all replicas in CrashLoopBackOff
- CAST AI agent pod not running
- Any pod Pending for more than 15 minutes
- Any WOOP recommendation exceeding 100 GiB memory or 100 CPU

cores for a single workload. This is ALWAYS a bug or runaway logic – no discovery-puller pod should ever need 10 TB of RAM. Check `woop_get_workloads` for the raw recommendation values.

- CAST AI agent pod restarting more than 3 times in the last hour, or showing `ExitCode:0` with `restartCount > 5`. Silent agent failures have caused multi-day outages for this customer.
- Cluster node count or total pod count increased by >50% vs. the trailing 1-hour average without a corresponding deployment event (cascading scaling / runaway autoscaler).

WARNING (post, but less urgent):

- OOMKill count increasing but below critical threshold
- Memory usage trending upward across 3+ consecutive snapshots without leveling off (potential leak)
- Workload with optimization enabled but no recommendation for >2 hours
- Node running at >90% memory utilization
- CAST AI agent pod `restartCount` between 1-3 in the last hour (early signal before it becomes critical)
- Workload autoscaler webhook not responding or exporter pod not Running (recommendations won't be applied even if correct)
- Any single pod memory usage exceeding 2x its request across 3+ snapshots (potential memory leak in the application itself, which will eventually trigger OOM recovery compounding)

HEALTHY (do not post):

- All pods Running with expected restart counts
- Resource usage within expected ranges
- Aggregator at zero replicas (this is normal)
- Sporadic single OOMKill with no repeat (transient)

Snapshot Data

{SNAPSHOT_JSON}

Slack Message Format

When the evaluation returns WARNING or CRITICAL, the agent formats a Slack message like this:

```
*CRITICAL – Grip Security prod-us-4*

*OOMKill spiral detected* in `cengage/discovery-puller`

• 6 OOMKills in the last hour (3x the threshold)
• Memory limit is 128 MiB but workload needs ~1.2 GiB
```

- WOOP recommended 1118 MiB but pod shows 128 MiB (mismatch)

***Suggested action:** Disable WOOP for this workload immediately and investigate why the applied limit diverges from the recommendation.

Snapshot taken at 2026-06-03T14:35:00Z — next check in 5 min

Example for the absurd recommendation scenario (June 1 incident):

 ***CRITICAL — Grip Security prod-us-4***

Absurd WOOP recommendation on `athenahealth/discovery-puller`

- WOOP is recommending 10 TB memory request for this workload
- No EC2 instance can satisfy this — pods will be unschedulable
- Root cause: OOM recovery overhead compounded with Max Usage strategy after a single pod spiked from 1 GiB to ~2 TiB
- 2 pods currently Pending (insufficient resources)

***Suggested action:** Disable WOOP optimization for this workload immediately. Investigate the memory spike on the outlier pod — likely a memory leak in the application. Consider switching from Max Usage to P99 strategy to prevent outlier pods from driving recommendations.

Snapshot taken at 2026-06-01T18:55:00Z — next check in 5 min

Incident History: What the Watchdog Would Have Caught

Review of the #castai_grip_security_ext Slack channel reveals a pattern of recurring incidents going back to January 2026. The table below maps each incident to the watchdog detection rule that would have caught it, and estimates how much earlier detection would have occurred.

Date	Incident	Watchdog Rule	Actual Detection	Watchdog Detection
Jun 1	discovery-puller requesting 10TB RAM in athenahealth (WOOP OOM recovery overshoot)	absurd_recommendation: WOOP recommendation >100 GiB	Customer reported	~5 min (first snapshot cycle)
May 21-25	Int64 overflow: 128 MiB limit applied instead of 143 GiB, sustained OOMKills for 4 days	mismatch: applied vs. intended >50%, plus oomkill: >3 OOMKills/hour	Customer reported after 4 days	~5 min
May 25	CAST AI agent restarting 11+ times on prod-us-3 with route-to-host errors	agent_restart: restartCount >3/hour	Customer reported	~5 min
May 18	CAST AI agent down across all production clusters	agent: agent pod not Running	Customer reported (urgent)	~5 min
Apr 3	Cascading scaling loop: double expected nodes, ~3000 pods	cascading_scaling: node/pod count >50% above trailing average	Customer found after ~3 hours	~10 min (2 cycles to confirm trend)
Apr 1	WA webhook non-functional on prod-eu; exporter metric limit hit on prod4	webhook_failure: WA exporter/webhook pods unhealthy	CAST AI proactive audit	~5 min
Mar 29	Workloads not scaling, stuck with OOMKills and throttling	oomkill + data_gap: OOMKills with no active recommendation	Customer reported	~5 min
Mar 18	castai-agent exiting with ExitCode:0 silently for days (122-reply thread)	agent_restart: ExitCode:0 with restartCount >5	Customer reported after days	~5 min
Jan 13	castai-agent OOMing itself, needed 25 GB RAM, showing as unresponsive	agent: agent pod OOMKilled or not Running	Customer reported	~5 min

In every case except the April 1 webhook audit (which CAST AI caught proactively), the customer detected the issue before us. The watchdog would have inverted this dynamic entirely — detection within minutes instead of hours or days, with the customer receiving a diagnostic before they even notice the problem.

Implementation Notes

The SE should implement this as a single Python service (or a set of scripts triggered by cron/K8s CronJob) with three modules: collector, evaluator, and notifier. The collector is pure API calls and kubectl, the evaluator is a single Anthropic API call, and the notifier is a Slack webhook post. Total code should be under 500 lines.

Collector module: Uses the CAST AI API Python SDK and kubectl with the cluster's kubeconfig. Outputs a standardized JSON blob. Stores the last 12 snapshots in a local JSON file (or Redis/S3 if running in K8s). Each snapshot should include a UTC timestamp, pod list with status and resource specs, node list with capacity and utilization, WOOP recommendations with intended vs. applied values, and CAST AI agent pod status.

Evaluator module: Calls the Anthropic Messages API with the prompt above. Uses claude-haiku-4-5 for cost efficiency (~\$0.001 per evaluation at ~4K input tokens per snapshot). Falls back to claude-sonnet-4-5 if we want higher fidelity during the initial tuning period. Parses the JSON response and passes it to the notifier.

Notifier module: Posts to Slack via incoming webhook or the Slack API. Formats the findings into the message template shown above. Implements a deduplication window: if the same finding (same category + same workload) was posted in the last 30 minutes, skip it to avoid flooding the channel. Posts the daily summary at 08:00 UTC regardless.

Baseline tuning: During the first 24 hours, run the agent in dry-run mode: evaluate but don't post to Slack. Review the output to calibrate the baseline context in the prompt (the "known baseline behaviors" section). This is critical—without it, scale-to-zero workloads will trigger false positives on every cycle.

Open Questions

First, should this run inside the customer's cluster or as a CAST AI-hosted service? Running inside the cluster gives direct kubectl access but adds another thing we maintain in their environment. Running externally means we need a kubeconfig with read-only access and API calls go over the network. Second, do we want the LLM evaluation to have access to CAST AI internal state beyond the API (e.g., Workload Autoscaler logs, mutation webhook logs)? This would catch issues like the safety-mechanism gap faster but requires more plumbing. Third, who owns the Slack channel—is this a shared channel between CAST AI and Grip, or an internal CAST AI channel where we triage before escalating? The onsite meeting implied a shared channel. Fourth, should we generalize this for other customers immediately, or build it bespoke for Grip and generalize later? Recommendation is bespoke first—the baseline context needs to be per-cluster.

Success Metrics

The agent should detect any OOMKill spiral within 10 minutes of onset (2 evaluation cycles). False positive rate should be below 1 alert per day during normal operations after the 24-hour tuning period. Mean time to customer awareness of an issue should drop from days (current state) to under 15 minutes. Grip Security should report that they feel proactively supported rather than reactive within two weeks of deployment.

Next Steps

Owner	Action	Due	Status
SE (assigned)	Implement collector module and validate API/kubectl data shape against a live snapshot from prod-us-4	June 5	Not started
SE (assigned)	Implement evaluator module with the sample prompt, run in dry-run mode for 24 hours	June 6	Not started

Leon / TAM	Review dry-run output and tune baseline context to reduce false positives	June 7	Not started
SE (assigned)	Implement notifier module, connect to shared Slack channel, go live	June 8	Not started
Guy / TAM	Share the monitoring plan with Grip Security and confirm Slack channel setup	June 4	Not started

Appendix: Incident Summary

This section provides a condensed summary of the two known incidents that motivated this design.

Incident 1: Integer Overflow OOMKill (May 21–25)

The Workload Autoscaler computed a memory limit of $143 \text{ GiB} \times 100$ (the configured multiplier), producing ~14 TiB. This value overflowed the int64 representation in millibytes and wrapped to 128 MiB. Because Kubernetes clamps requests to not exceed limits, the memory request was also set to 128 MiB. The workload (discovery-puller in cengage and williamsmullen namespaces), which requires 800 MB–1.5 GB per pod, entered a continuous OOMKill loop. The safety mechanism (disable after 20 OOMKills/hour) never triggered because the mismatch between applied and intended values prevented OOM events from being counted. The issue persisted for approximately 4 days until Grip Security reported it. Fix shipped in Workload Autoscaler v1.0.25.

Incident 2: OOMKill Recovery Overshoot (May 31–June 1)

The OOMKill recovery mechanism increased resource requests to values larger than any available instance type, causing affected workloads to become unschedulable. Details are in the second RCA shared internally. This incident occurred while trust was already low from Incident 1, and was the direct catalyst for the June 1 onsite meeting with Grip Security's engineering leadership.

Slack Context

The internal thread (C0AAF8XJYQZ, June 1–3) captures the onsite meeting summary from Guy and Shahar: the customer's CTO is questioning CAST AI's value, they want a shared spreadsheet of improvements, and the team agreed that alerting and monitoring is the first priority. Sandra noted the engineering team is investigating whether the aggregator workload's unbounded memory growth is a memory leak in the application itself.